

安全漏洞审计报告 · 第5轮

用户信息管理平台 文件包含漏洞审计报告

User Management Platform — File Inclusion Vulnerability Assessment

目标系统：用户信息管理平台 (Flask)

报告日期：2026-07-13

漏洞等级：严重 × 2

修复状态：✅ 全部已修复

目录

1. 报告概述
2. 漏洞详情
3. 攻击演示（含实况结果）
4. 修复方案
5. 修复前后对照
6. 修复验证
7. 全部五轮汇总

1. 报告概述

1.1 本轮背景

系统新增了动态页面加载功能（`/page?name=help`），该功能读取 `pages/` 目录下的 HTML 文件并显示在首页上。由于代码直接将用户输入的 `name` 参数拼接到文件路径中，且没有做任何校验，导致严重的本地文件包含（**LFI**）漏洞。

1.2 什么是文件包含漏洞？

文件包含漏洞（**File Inclusion**）是指 Web 应用将用户输入作为文件路径来读取或执行文件。攻击者可利用 `../` 跳出限制目录，读取服务器上的任意文件（如源码、密码文件、数据库）。这是 OWASP Top 10 常见漏洞之一。

2. 漏洞详情

VULN-19 : 本地文件包含 (LFI)

● 严重

位置 : /page — GET 路由

漏洞代码 :

```
name = request.args.get("name", "")
page_path = os.path.join("pages", name)  ← 直接拼接用户输入
open(page_path, "r").read()  ← 读取任意文件
```

描述 : 攻击者输入 ../app.py , 路径变为 pages/../app.py , 等价于读取 app.py。输入 ../../../../etc/passwd 可读取系统文件。

VULN-20 : 敏感文件泄露

● 严重

描述 : 利用文件包含可读取以下敏感文件 :

- app.py — Flask 源码泄露 (数据库配置、密钥逻辑)
- data/users.db — SQLite 数据库 (所有用户名密码)
- .git/config — Git 仓库配置
- /etc/passwd — 系统用户列表
- /etc/shadow — 系统密码哈希 (可暴力破解)

3. 攻击演示

3.1 读取 **app.py** 源码（文件包含 + 路径遍历）

攻击请求

GET /page?name=../app.py

攻击结果

成功读取 Flask 应用源码：

```
from flask import Flask, render_template
import sqlite3
app.secret_key = os.environ.get("SECRET_KEY", ...)
```

3.2 读取系统密码文件

攻击请求

GET /page?name=../../../../etc/passwd

攻击结果

成功读取系统用户列表：

```
root:x:0:0:root:/root:/usr/bin/zsh
```

3.3 读取密码哈希

攻击请求

GET /page?name=../../../../etc/shadow

攻击结果

成功读取到密码哈希，可离线暴力破解：

```
root:$y$j9T$pl0EafxSTnm.o4cBGWzZL1$...
```

3.4 读取 **Git** 配置

攻击请求

GET /page?name=../.git/config

攻击结果

成功读取仓库配置：

```
[core]
repositoryformatversion = 0
```

4. 修复方案

核心修复：路径规范化 + 目录边界检查

```
# 修复文件包含漏洞的关键：检查文件是否在允许的目录内
pages_dir = os.path.join(os.path.dirname(__file__), "pages")
requested_path = os.path.abspath(os.path.join(pages_dir, name))

if not requested_path.startswith(os.path.abspath(pages_dir) + os.sep):
    page_content = "页面不存在" # 禁止读取 pages/ 目录外的文件
```

原理：无论攻击者传入多少 `../`，`os.path.abspath()` 都会解析出文件的真实路径。只要这个真实路径不在 `pages/` 目录下，就拒绝读取。这样既保留了动态页面功能，又防止了文件包含攻击。

5. 修复前后对照

✗ 修复前 (存在文件包含漏洞)

```
page_path = os.path.join("pages", name)
open(page_path).read()
```

- name = "../app.py"
- 读取 pages/../app.py
- ☒ 成功包含源码

✓ 修复后 (文件包含防御)

```
requested_path = os.path.abspath(...)
if not in pages_dir: reject
```

- name = "../app.py"
- 规范化后不在 pages/ 下
- ☐ 拒绝包含

6. 修复验证

#	测试	结果
1	正常访问 ?name=help → 显示帮助中心	✓ 通过
2	文件包含 ?name=../app.py → 被拦截	✓ 通过
3	文件包含 ?name=../../etc/passwd → 被拦截	✓ 通过
4	文件包含 ?name=../../etc/shadow → 被拦截	✓ 通过
5	文件包含 ?name=../.git/config → 被拦截	✓ 通过

7. 全部五轮汇总

轮次	审计内容	漏洞数	严重	高危	中危	状态
第 1 轮	密码安全	8	4	-	4	✓
第 2 轮	SQL 注入	2	2	-	-	✓
第 3 轮	文件上传	1	1	-	-	✓
第 4 轮	业务逻辑	7	2	2	3	✓
第 5 轮	文件包含	2	2	-	-	✓
合计		20	11	2	7	全部已修复

安全建议

- 永远不要信任用户输入的文件名：任何文件操作前必须校验路径
- 使用白名单：只允许预定义的页面名称，禁止用户自定义路径
- 目录边界检查：用 `os.path.abspath()` 解析真实路径，确保在允许范围内
- 最小权限原则：Web 应用不应有读取系统文件（如 `/etc/shadow`）的权限

安全漏洞审计报告 · 第5轮 — 文件包含安全 | 报告日期：2026-07-13

项目地址：<https://github.com/sencly-book/user-management>